VERILOG MINOR PROJECT: SOURCE CODE

Thipperudra K T | Roll No. 4SM23EE037 | Unlox Academy VLSI September 2026 Batch | 31 October 2026

Program 1: Two reg variables

Aim: Declare two reg variables, assign values in an initial block and display them with $display.

```
`timescale 1ns / 1ps
module program1;
    reg [7:0] a, b;               // two 8-bit reg variables
    initial begin
        a = 8'd25;                // decimal 25
        b = 8'hA5;                // hexadecimal A5 = decimal 165
        $display("a = %0d (binary %b, hex %h)", a, a, a);
        $display("b = %0d (binary %b, hex %h)", b, b, b);
    end
endmodule
```

Expected output:

```
a = 25 (binary 00011001, hex 19)
b = 165 (binary 10100101, hex a5)
```

Program 2: Integer addition

Aim: Declare integer variables, add two numbers and display the result.

```
module program2;
    integer num1, num2, sum;      // 32-bit signed variables
    initial begin
        num1 = 15;
        num2 = 27;
        sum  = num1 + num2;       // arithmetic addition
        $display("num1 = %0d, num2 = %0d", num1, num2);
        $display("Sum  = %0d + %0d = %0d", num1, num2, sum);
    end
endmodule
```

Expected output:

```
num1 = 15, num2 = 27
Sum  = 15 + 27 = 42
```

Program 3: AND operation using assign

Aim: Declare a wire and implement an AND operation using the assign statement.

```
`timescale 1ns / 1ps
module program3;
    reg  a, b;                    // inputs driven in the initial block
    wire y;                       // net driven by assign
    assign y = a & b;             // continuous assignment: AND gate
    initial begin
        $monitor("time=%0d a=%b b=%b y=%b", $time, a, b, y);
        a = 0; b = 0;
        #10 a = 0; b = 1;
        #10 a = 1; b = 0;
        #10 a = 1; b = 1;
        #10 $finish;
    end
endmodule
```

Expected output:

```
time=0 a=0 b=0 y=0
time=10 a=0 b=1 y=0
time=20 a=1 b=0 y=0
time=30 a=1 b=1 y=1
```

Program 4: reg, integer and time

Aim: Use reg, integer and time data types and display their values.

```
`timescale 1ns / 1ps
module program4;
    reg [3:0] r;                  // 4-bit reg
    integer   i;                  // 32-bit signed integer
    time      t;                  // 64-bit simulation time
    initial begin
        $display("Before assignment: r = %b, i = %0d", r, i);
        r = 4'b1010;
        i = -25;
        #50;                      // wait for 50 ns
        t = $time;                // capture current time
        $display("r = %b (binary) = %0d (decimal)", r, r);
        $display("i = %0d", i);
        $display("t = %0d ns", t);
    end
endmodule
```

Expected output:

```
Before assignment: r = xxxx, i = x
r = 1010 (binary) = 10 (decimal)
i = -25
t = 50 ns
```

Program 5: OR gate with continuous assignment

Aim: Write a Verilog module for an OR gate using a continuous assignment.

```
`timescale 1ns / 1ps
module or_gate(input a, input b, output y);
    assign y = a | b;             // continuous assignment: OR gate
endmodule

module tb_or_gate;                // testbench
    reg  a, b;
    wire y;
    or_gate uut (.a(a), .b(b), .y(y));
    initial begin
        $monitor("time=%0d a=%b b=%b y=%b", $time, a, b, y);
        a = 0; b = 0;
        #10 a = 0; b = 1;
        #10 a = 1; b = 0;
        #10 a = 1; b = 1;
        #10 $finish;
    end
endmodule
```

Expected output:

```
time=0 a=0 b=0 y=0
time=10 a=0 b=1 y=1
time=20 a=1 b=0 y=1
time=30 a=1 b=1 y=1
```

How to Run (Icarus Verilog)

```
iverilog -o p1 src/program1.v && vvp p1
iverilog -o p5 src/program5.v && vvp p5
```

Summary

Table 1: Programs and results

Program	Concept	Result
1	reg with initial and $display	a = 25, b = 165
2	integer addition	15 + 27 = 42
3	wire with assign (AND)	y = 1 only for a = 1, b = 1
4	reg, integer, time	r = 1010, i = -25, t = 50
5	OR gate, continuous assignment	y = 0 only for a = 0, b = 0